



PROGRAMACIÓN

1º DAM

RETO 5



ENUNCIADO

Utilizando el SGBD adecuado:

- Crea una base de datos que se llame Ferretería
- Crea una tabla se llame articulos con 4 campos que todos sean con inserción obligatoria
 - **idArticulo**: clave primaria (no es autincremental)
 - **descripcion**: dato de tipo cadena
 - **pvp**: dato de tipo decimal
 - **stock**: dato de tipo entero
- Inserta un procedimiento almacenado que inserte un articulo pasándole los parámetros adecuados.
Ejemplo de llamada: **call insertarArticulo(4,'Martillo percutor', 1900.4,2)**
- Crear un usuario y contraseña segura con **GRANT ALL PRIVILEGES**

Realiza un programa en Java que haga lo siguiente:

- Prepara el proyecto para la gestión de la base de datos de Ferretería
- Realiza la conexión con el usuario y contraseña adecuada.
- Realiza modificaciones en los registros sin interacción con el usuario
 - Borra todos los registros
 - Inserta 3 registros directamente, utiliza el ignore en la sentencia INSERT para evitar problemas al principio
 - Muestra todos los registros de la tabla
 - Modifica un registro buscando por id
 - Muestra todos los registros de la tabla
- Crea un método de mostrar todos los registros (Muestra todos los registros de la tabla).
Tienes que pasar por parámetro la conexión
- Realiza modificaciones en los registros interactuando con el usuario:
 - Pide datos al usuario e inserta un registros
 - Muestra todos los registros de la tabla
 - Pide datos al usuario y muestra por ID
 - Pide datos al usuario y borra un registro por ID
 - Muestra todos los registros de la tabla
- Llama al procedimiento almacenado con datos que se pide al usuario

Hay que utilizar en todo momento try-with-resources

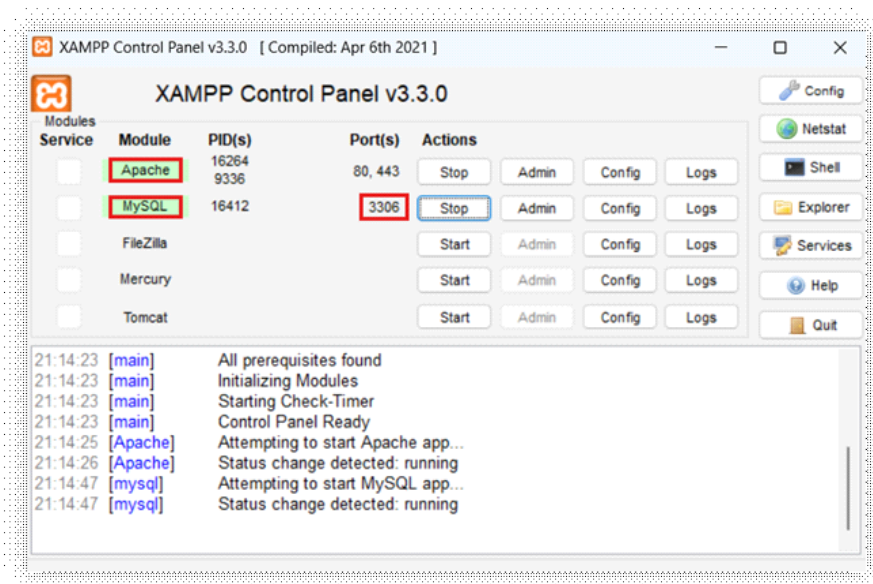


Utilizando el SGBD adecuado:

- Crea una base de datos que se llame Ferretería
- Crea una tabla se llame articulos con 4 campos que todos sean con inserción obligatoria
 - **idArticulo**: clave primaria (no es autincremental)
 - **descripcion**: dato de tipo cadena
 - **pvp**: dato de tipo decimal
 - **stock**: dato de tipo entero
- Inserta un procedimiento almacenado que inserte un articulo pasándole los parámetros adecuados.
Ejemplo de llamada: **call insertarArticulo(4,'Martillo percutor', 1900.4,2)**
- Crear un usuario y contraseña segura con **GRANT ALL PRIVILEGES**

RESOLUCIÓN

> Empleamos MySQL como SGBD. Con XAMPP levantamos el servidor Apache, conectándose a la base de datos a través del puerto 3306.



> Mediante **phpMyAdmin** creamos la base de datos y realizamos los demás puntos de este primer apartado con el siguiente conjunto de sentencias SQL:

> Creamos la BBDD FERRETERIA:

```
CREATE DATABASE FERRETERIA;
```

> Creamos la tabla ARTICULOS:

```
CREATE TABLE ARTICULOS(  
idArticulo INT PRIMARY KEY NOT NULL,  
descripcion VARCHAR(30) NOT NULL,  
pvp DECIMAL(5,1) NOT NULL,  
stock INT NOT NULL  
);
```



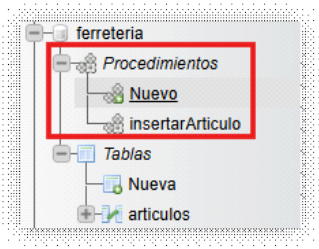
> **Procedimiento insertarArticulo:**

```
DELIMITER //

CREATE PROCEDURE insertarArticulo(
    IN p_idArticulo INT,
    IN p_descripcion VARCHAR(30),
    IN p_pvp DECIMAL(5,1),
    IN p_stock INT
)
BEGIN
    INSERT INTO ARTICULOS (idArticulo, descripcion, pvp, stock)
    VALUES (p_idArticulo, p_descripcion, p_pvp, p_stock);
END //

DELIMITER ;
```

> En el panel izquierdo de phpMyAdmin podemos ver que la BBDD creada contiene la tabla y el procedimiento creados:



> **Ejecutamos el procedimiento:**

```
CALL insertarArticulo(1, 'Martillo percutor', 1900.4, 2);
```

> Comprobamos que funciona visualizando el contenido de la tabla:

```
SELECT * FROM articulos;
```

idArticulo	descripcion	pvp	stock
1	Martillo percutor	1900.4	2

> **Creamos un usuario y contraseña segura con GRANT ALL PRIVILEGES:**

```
GRANT ALL PRIVILEGES ON FERRETERIA.* TO 'fran79'@localhost IDENTIFIED BY 'fran79*';
```

> Para visualizar que se ha creado correctamente podemos probar:

```
-- Muestra todos los usuarios que hay en la BD de MySQL:
SELECT * FROM mysql.user;
```

Host	User	P
localhost	root	
%	usuariocoches *E	
127.0.0.1	root	
::1	root	
localhost	pma	
localhost	usuariocoches *E	
localhost	fran79	1



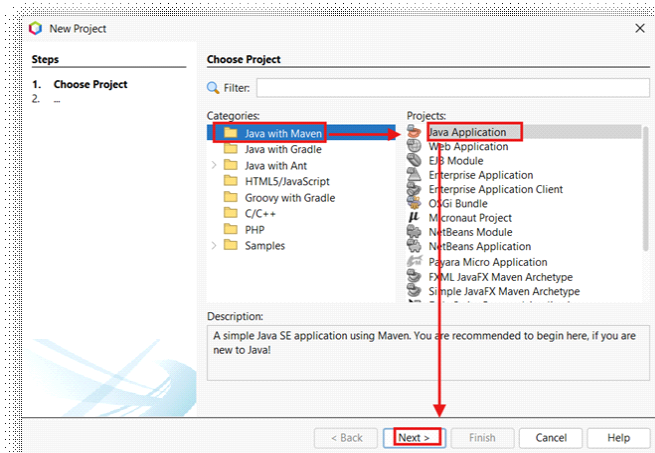
Realiza un programa en Java que haga lo siguiente:

- Prepara el proyecto para la gestión de la base de datos de Ferretería
- Realiza la conexión con el usuario y contraseña adecuada.
- Realiza modificaciones en los registros sin interacción con el usuario
- Borra todos los registros
- Inserta 3 registros directamente, utiliza el ignore en la sentencia INSERT para evitar problemas al principio
- Muestra todos los registros de la tabla
- Modifica un registro buscando por id
- Muestra todos los registros de la tabla
- Crea un método de mostrar todos los registros (Muestra todos los registros de la tabla). Tienes que pasar por parámetro la conexión
- Realiza modificaciones en los registros interactuando con el usuario:
 - Pide datos al usuario e inserta un registros
 - Muestra todos los registros de la tabla
 - Pide datos al usuario y muestra por ID
 - Pide datos al usuario y borra un registro por ID
 - Muestra todos los registros de la tabla
- Llama al procedimiento almacenado con datos que se pide al usuario

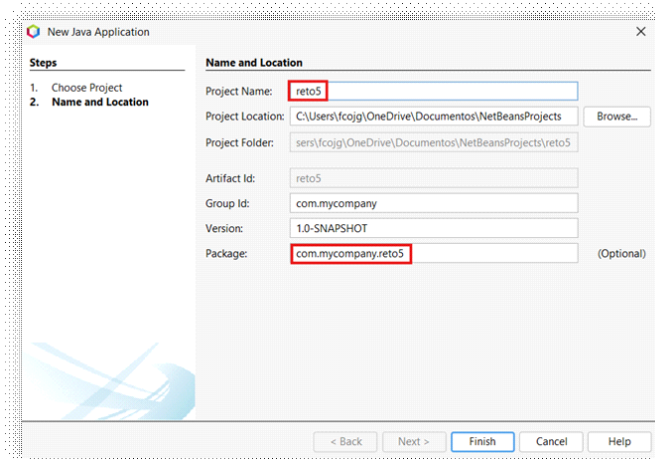
Hay que utilizar en todo momento try-with-resources

> Preparamos el proyecto para la gestión de la base de datos de Ferretería:

> Elegimos un proyecto tipo MAVEN para poder usar el repositorio que permita la conexión al SGBD (MySQL):

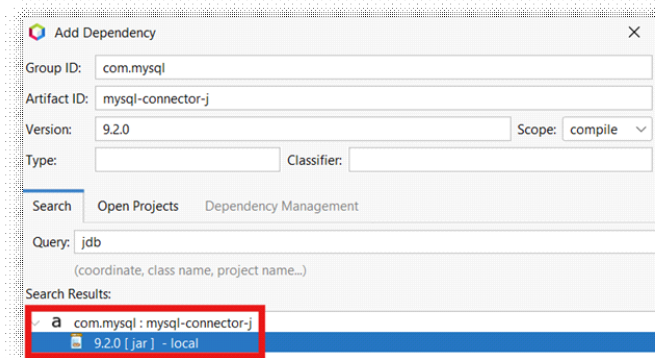


> Le damos nombre:

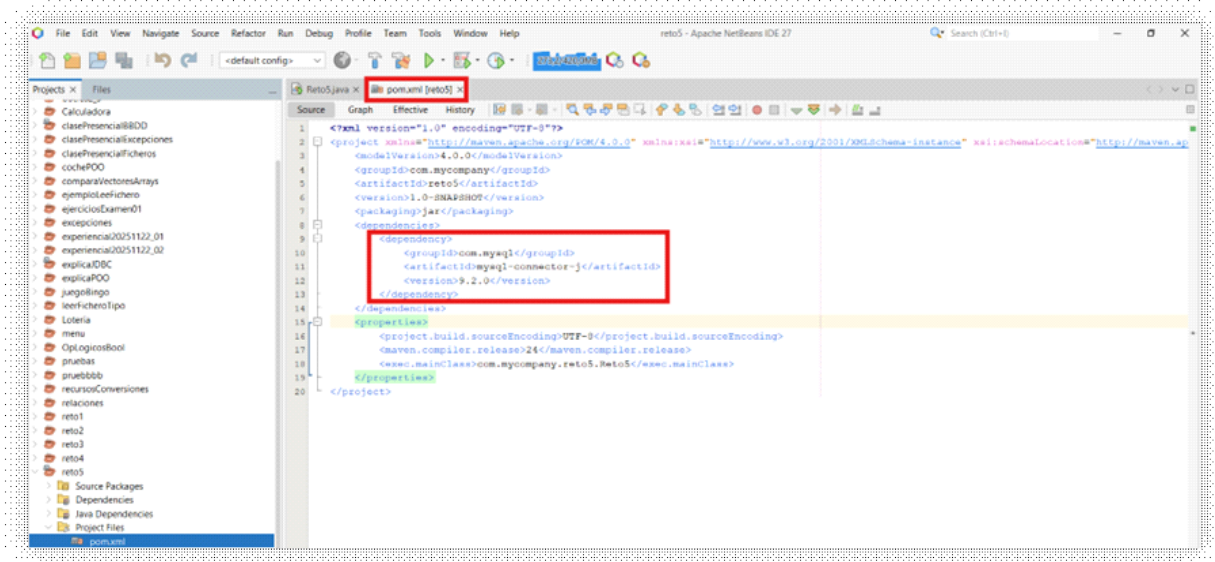




> Agregamos las dependencias necesarias para poder conectar al SGBD (MySQL) desde el repositorio de MAVEN <https://mvnrepository.com/>



> Observamos como se han incluido en el archivo pom.xml del proyecto:



> Ahora construimos el proyecto (sobre el archivo de proyecto, botón derecho → clean && build), lo que instala los drivers adecuados con los que ya podemos hacer la conexión a la BBDD.

> **Realiza la conexión con el usuario y contraseña adecuada:**

> Los parámetros a emplear para la conexión, con el usuario y contraseña creados anteriormente son:

```
//Parámetros conexión con la BBDD
String url = "jdbc:mysql://localhost:3306/ferreteria";
String user = "fran79";
String password = "fran79*";
```

> La conexión se realiza dentro de los paréntesis del try; esto, denominado try-catch-with-resources permite que la conexión y el resto de recursos que se abran entre estos paréntesis se cierren después de ejecutar el try automáticamente (posible desde Java 7).

```
try (Connection con = DriverManager.getConnection(url, user, password); {
    //Operaciones a realizar
} catch (SQLException e) {
    System.out.println("Conexión no correcta / Error SQL");
    System.out.println(e.getMessage());
}
```

> Para ver el código completo ir al último apartado.



> Realiza modificaciones en los registros sin interacción con el usuario:

- > Empleamos Statements definidos en el try a partir del método **.executeUpdate()** para los casos de INSERT/UPDATE/DELETE y el método **.executeQuery()** para consultas SELECT, pasando como argumento la sentencia SQL a ejecutar en cada caso.
- > En el caso de inserción de 3 registros seguidos se emplea la sintaxis **INSERT IGNORE INTO**, que permite que el proceso no se detenga por completo si ocurre algún tipo de error "ignorable" (como un duplicado, valores nulos o de tipo erróneo...).
- > Para mostrar todos los registros de la tabla se emplea un ResultSet que recoge la consulta que arroja el SELECT, que se realiza con un **.executeQuery()**.
- > Ver código completo comentado en el último apartado.

> Crea un método que muestre todos los registros de la tabla, pasándole como parámetro la conexión:

- > Se crea un método dentro de la clase principal (Reto5) que llamamos **mostrarRegistros**.
- > Se le pasa la conexión que ya ha sido creada y comprobada (cuando se llame al método desde el main) en el primer try.
- > No obstante se manejan los errores correspondientes a la sentencia de consulta (statement) declarándola en los paréntesis del try.
- > Quedaría como sigue:

```
public static void mostrarRegistros(Connection conexion){

    try (Statement stmt = conexion.createStatement()){

        //Sentencia SQL para mostrar todos los registros
        String sqlSelect = "SELECT * FROM ARTICULOS";
        //Guardamos la consulta en un cursor
        ResultSet rs = stmt.executeQuery(sqlSelect);
        System.out.println("TABLA ARTICULOS");
        System.out.println("-----");

        while (rs.next()) {
            System.out.println("idArticulo: "+rs.getInt(1)+" , descripcion: "+rs.getString(2)+
                " , pvp: "+rs.getDouble(3)+" , stock: "+rs.getInt(4));
        }
    } catch (SQLException e) {
        System.out.println("Conexión no correcta / Error SQL");
        System.out.println(e.getMessage());
    }

}
```



> **Realiza modificaciones en los registros interactuando con el usuario:**

> Empleamos PreparedStatement definidos en el try a partir del método **.executeUpdate()** para los casos de INSERT/UPDATE/DELETE y el método **.executeQuery()** para consultas SELECT, pasando como argumento la sentencia SQL a ejecutar en cada caso.

> Inicialmente y antes del try se declaran los String que se van a pasar a las PreparedStatement para cada consulta:

```
String sqlInsertaP = "INSERT INTO ARTICULOS (idArticulo,descripcion,pvp,stock) VALUES (?, ?, ?, ?)";  
String sqlSelectIdP = "SELECT * FROM ARTICULOS WHERE idArticulo = ?";  
String sqlBorrado = "DELETE FROM ARTICULOS WHERE idArticulo = ?";
```

> A estas se le pasarán como parámetros los valores tomados por teclado a través de los métodos **.setInt()**, **.setString()**...

> Para cada consulta diferente se declara una PreparedStatement en el try:

```
try (Connection con = DriverManager.getConnection(url, user, password);  
    Statement sentencia = con.createStatement();  
  
    PreparedStatement sentenciaPreparada1 = con.prepareStatement(sqlInsertaP);  
    PreparedStatement sentenciaPreparada2 = con.prepareStatement(sqlSelectIdP);  
    PreparedStatement sentenciaPreparada3 = con.prepareStatement(sqlBorrado);){  
    ...  
}
```

> Ver código completo comentado en el último apartado.

> **Llama al procedimiento almacenado con datos que se pide al usuario:**

> La llamada a procedimientos se realiza mediante **CallableStatement**:

```
try (Connection con = DriverManager.getConnection(url, user, password);  
    ...  
    CallableStatement cstmt = con.prepareCall(sqlProcedimiento);){  
    ...  
}
```

> Previamente se debe haber definido antes del try la sentencia correspondiente:

```
String sqlProcedimiento = "{call insertarArticulo(?, ?, ?, ?)}";
```

> Funciona de manera similar a las PreparedStatement, pudiendo pasarle los valores que serán parámetro del procedimiento.



```
package com.mycompany.reto5;

import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Scanner;

public class Reto5 {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);

        //Parámetros conexión con la BBDD
        String url = "jdbc:mysql://localhost:3306/ferreteria";
        String user = "fran79";
        String password = "fran79*";
        String sqlInsertaP = "INSERT INTO ARTICULOS (idArticulo,descripcion,pvp,stock) VALUES (?, ?, ?, ?)";
        String sqlSelectIdP = "SELECT * FROM ARTICULOS WHERE idArticulo = ?";
        String sqlBorraId = "DELETE FROM ARTICULOS WHERE idArticulo = ?";
        String sqlProcedimiento = "{call insertarArticulo(?, ?, ?, ?)}";

        try (Connection con = DriverManager.getConnection(url, user, password);
            Statement sentencia = con.createStatement();
            PreparedStatement sentenciaPreparada1 = con.prepareStatement(sqlInsertaP);
            PreparedStatement sentenciaPreparada2 = con.prepareStatement(sqlSelectIdP);
            PreparedStatement sentenciaPreparada3 = con.prepareStatement(sqlBorraId);
            CallableStatement cstmt = con.prepareCall(sqlProcedimiento);){

            //Modificaciones sin interacción con el usuario

            //Sentencia SQL para borrar todos los registros
            String sqlBorraTodo = "DELETE FROM ARTICULOS";
            //Ejecución
            sentencia.executeUpdate(sqlBorraTodo);
            System.out.println("Borrado hecho");

            //Sentencia SQL para insertar 3 registros seguidos
            String sqlInserta = ""
                INSERT IGNORE INTO ARTICULOS (idArticulo,descripcion,pvp,stock)
                VALUES (2, "Brocas 4 puntas", 355.45, 4),
                (3, "Amoladora a batería", 638.99, 3),
                (4, "Maletín de herramientas", 28.85, 7)
                """;
            //Ejecución
            sentencia.executeUpdate(sqlInserta);
            System.out.println("Inserto hecho");
```





```
System.out.println("TABLA ARTICULOS");
System.out.println("-----");

//Sentencia SQL para mostrar todos los registros
String sqlSelect = "SELECT * FROM ARTICULOS";
//Guardamos la consulta en un cursor
ResultSet rs = sentencia.executeQuery(sqlSelect);

while (rs.next()) {
    System.out.println("idArticulo: "+rs.getInt(1)+", descripcion: "+rs.getString(2)+
        ", pvp: "+rs.getDouble(3)+" , stock: "+rs.getInt(4));
}
System.out.println("");

//Llamada al método creado para enseñar todos los registros
mostrarRegistros(con);

//Modificaciones CON interacción con el usuario

//Pide datos e ingresa un registro
System.out.println("Introduzca datos para un registro (id, descripcion, pvp, stock): ");
int id = teclado.nextInt();
String descripcion = teclado.next();
double pvp = teclado.nextDouble();
int stock = teclado.nextInt();

sentenciaPreparada1.setInt(1, id);
sentenciaPreparada1.setString(2, descripcion);
sentenciaPreparada1.setDouble(3, pvp);
sentenciaPreparada1.setInt(4, stock);
sentenciaPreparada1.executeUpdate();

//Muestra todos los registros
mostrarRegistros(con);

//Pide id y muestra registro
System.out.println("Introduzca un id para mostrar el registro: ");
id = teclado.nextInt();
System.out.println("");

sentenciaPreparada2.setInt(1, id);
rs = sentenciaPreparada2.executeQuery();
while (rs.next()) {
    System.out.println("idArticulo: "+rs.getInt(1)+", descripcion: "+rs.getString(2)+
        ", pvp: "+rs.getDouble(3)+" , stock: "+rs.getInt(4));
}
System.out.println("");

//Pide id y borra registro
System.out.println("Introduzca un id para borrar el registro: ");
id = teclado.nextInt();
System.out.println("");

sentenciaPreparada3.setInt(1, id);
sentenciaPreparada3.executeUpdate();
```





```
//Muestra todos los registros
mostrarRegistros(con);

//Llama al procedimiento almacenado con parámetros pedidos al usuario
System.out.println("Introduzca datos para un registro mediante llamada a procedimiento (id, descripcion, pvp, stock): ");
id = teclado.nextInt();
descripcion = teclado.next();
pvp = teclado.nextDouble();
stock = teclado.nextInt();
System.out.println("");

cstmt.setInt(1, id);
cstmt.setString(2, descripcion);
cstmt.setDouble(3, pvp);
cstmt.setInt(4, stock);
cstmt.execute();

//Muestra todos los registros
mostrarRegistros(con);

} catch (SQLException e) {
    System.out.println("Conexión no correcta / Error SQL");
    System.out.println(e.getMessage());
}

}

}

public static void mostrarRegistros(Connection conexion){

    try (Statement stmt = conexion.createStatement();){

        //Sentencia SQL para mostrar todos los registros
        String sqlSelect = "SELECT * FROM ARTICULOS";
        //Guardamos la consulta en un cursor
        ResultSet rs = stmt.executeQuery(sqlSelect);
        System.out.println("TABLA ARTICULOS mediante método");
        System.out.println("-----");

        while (rs.next()) {
            System.out.println("idArticulo: "+rs.getInt(1)+", descripcion: "+rs.getString(2)+
                ", pvp: "+rs.getDouble(3)+", stock: "+rs.getInt(4));
        }
        System.out.println("");

    } catch (SQLException e) {
        System.out.println("Conexión no correcta / Error SQL");
        System.out.println(e.getMessage());
    }

}

}
```



COMPROBACIÓN DE FUNCIONAMIENTO

```
--- exec:3.1.0:exec (default-cli) @ reto5 ---
Borrado hecho
Inserto hecho
TABLA ARTICULOS
-----
idArticulo: 2, descripcion: Brocas 4 puntas, pvp: 355.5, stock: 4
idArticulo: 3, descripcion: Amoladora a bater a, pvp: 639.0, stock: 3
idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7

TABLA ARTICULOS mediante m todo
-----
idArticulo: 2, descripcion: Brocas 4 puntas, pvp: 355.5, stock: 4
idArticulo: 3, descripcion: Amoladora a bater a, pvp: 639.0, stock: 3
idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7

Introduzca datos para un registro (id, descripcion, pvp, stock):
5
Martillo
12,99
8
TABLA ARTICULOS mediante m todo
-----
idArticulo: 2, descripcion: Brocas 4 puntas, pvp: 355.5, stock: 4
idArticulo: 3, descripcion: Amoladora a bater a, pvp: 639.0, stock: 3
idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7
idArticulo: 5, descripcion: Martillo, pvp: 13.0, stock: 8

Introduzca un id para mostrar el registro:
4

idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7

Introduzca un id para borrar el registro:
2

TABLA ARTICULOS mediante m todo
-----
idArticulo: 3, descripcion: Amoladora a bater a, pvp: 639.0, stock: 3
idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7
idArticulo: 5, descripcion: Martillo, pvp: 13.0, stock: 8

Introduzca datos para un registro mediante llamada a procedimiento (id, descripcion, pvp, stock):
6
Mesa
78,95
3

TABLA ARTICULOS mediante m todo
-----
idArticulo: 3, descripcion: Amoladora a bater a, pvp: 639.0, stock: 3
idArticulo: 4, descripcion: Malet n de herramientas, pvp: 28.9, stock: 7
idArticulo: 5, descripcion: Martillo, pvp: 13.0, stock: 8
idArticulo: 6, descripcion: Mesa, pvp: 79.0, stock: 3

-----
BUILD SUCCESS
-----
```